

Actividad IA num 1 : Semana 5

Escenario (Problema)	Algoritmo	Justificación (Por qué este)	Desventaja de usar el OTRO algoritmo
1. Redes Sociales Encontrar "amigos de amigos" (conexiones de 2° grado) en una red como LinkedIn o Facebook.	BFS (Búsqueda en Amplitud)	BFS explora "capa por capa". Garantía: Encuentra todos los nodos a distancia 1 (amigos directos), luego todos los nodos a distancia 2 ("amigos de amigos"). Es la forma más eficiente y directa de encontrar nodos a una distancia K específica.	DFS se iría "profundo" por una sola rama de conexión (ej. encontraría al amigo del amigo del amigo... Nivel 5) antes de haber encontrado a todos tus amigos de Nivel 2. Sería muy ineficiente.
2. Rutas GPS (sin ponderar) Encontrar la ruta con el menor número de paradas/calles entre dos puntos (asumiendo que cada calle "vale" lo mismo).	BFS	Garantía: BFS siempre encuentra el camino más corto (en número de aristas/paradas) en un grafo no ponderado. Como expande uniformemente desde el inicio, la primera vez que llega al destino, lo hace por la ruta más corta posible.	DFS encontraría un camino, pero casi con seguridad no sería el más corto. Podría tomar una ruta larga de 30 calles simplemente porque fue la primera que exploró a fondo, ignorando una ruta directa de 10 calles.
3. Resolver un Laberinto Encontrar cualquier camino válido desde la entrada (A) hasta la salida (B) de un laberinto complejo.	DFS (Búsqueda en Profundidad)	Eficiencia de memoria: DFS solo necesita almacenar el camino actual en la pila (memoria $O(\text{profundidad})$). Es ideal para explorar un camino hasta el final. Si falla, retrocede (backtracking) y prueba otro. Es como poner la mano en la pared y seguirla.	BFS consumiría muchísima memoria. Si el laberinto tiene muchas bifurcaciones (es "ancho"), la cola de BFS crecería exponencialmente, almacenando todas las posibles rutas simultáneamente.
4. Compiladores y Dependencias Detectar dependencias circulares en un sistema (ej. un compilador que ve A depende de B, y B depende de A).	DFS	Detección de ciclos: DFS es natural para detectar ciclos. Al explorar "profundo", puede detectar una "arista hacia atrás" (back edge), es decir, un nodo que apunta a otro que ya está en la pila de recursión actual.	BFS no es bueno para detectar ciclos en grafos dirigidos. No mantiene un registro del "camino actual", solo de las capas de distancia, por lo que no puede saber si una arista apunta a un ancestro en la misma ruta.
5. Búsqueda en Sistema de Archivos Encontrar un archivo específico (ej. mi_reporte.pdf) dentro de un árbol de directorios (carpetas dentro de carpetas).	DFS	Estructura del problema: Un sistema de archivos es un árbol (naturalmente profundo, no excesivamente ancho). DFS imita cómo un humano buscaría: abre una carpeta, luego una subcarpeta, y sigue así hasta el fondo antes de retroceder. Su uso de memoria $O(\text{profundidad})$ es mínimo.	BFS sería muy poco intuitivo y lento. Abriría todos los directorios del Nivel 1, luego todas las subcarpetas del Nivel 2, etc. Consumiría mucha memoria (cola) y realizaría muchos saltos de disco innecesarios.

Reflexion post IA

- ¿Los escenarios del IA coinciden con tu intuición?

Si justo como me lo imagine, uno sirve para busqueda en profundidad por una sola rama o "camino", mientras que el otro sirve para encontrar vecinos del mismo nivel.

- ¿Identificaste algún caso donde ambos algoritmos son igualmente válidos?

En los casos plasmados aqui no, cada uno tiene su caso particular

- ¿Qué criterio adicional no mencionó la IA que consideras importante?

Pues honestamente, los criterios de la IA son los mas importantes para el algoritmo, complejidad espacial y temporal, quizas falto que profundizara un poco mas el "como" elegir el algoritmo adecuado segun el problema, pero el prompt tampoco lo especifico.